SIMATS ENGINEERING
ASSESSMENT TOOL 5

COURSE NAME: Database Management Systems
COURSE CODE: CSA0506

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr. Hemavathi.R

QUESTIONS	Total Marks: 50		
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	**5**
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	**5**
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	**5**
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	**5**
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	**5**
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	**5**
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	**5**
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with	BL3 – Apply	**5**

	integrity constraint enforcement for a given scenario.		
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	**5**
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	**5**

Code of Conduct:

I G.JEEVESH(192511440) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G.Jeevesh

1.Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.

Assume:

Employee(EmpID, EmpName, DeptID, Salary)

EmpID	**EmpName**	**DeptID**	**Salary**
101	Rahul	D1	60000
102	Priya	D2	45000
103	Amit	D1	55000

Assume:

Employee(EmpID, EmpName, DeptID, Salary)

DeptID	***DeptName***
D1	***IT***
D2	***HR***

(a) Selection (σ)
Employees earning more than 50000

$$\sigma_{Salary>50000}(Employee)$$

(b) Projection (π)
Display employee names only

$$\pi_{EmpName}(Employee)$$

(c) Union (∪)
Employees from IT and HR

$$ITEmployees \cup HREmployees$$

(d) Set Difference (−)
Employees not working in IT

(e) Cartesian Product (×) $Employee - \sigma_{DeptID='D1'}(Employee)$

(f) Join (⋈) $Employee \times Department$

Employee details with department names

$$Employee \bowtie_{Employee.DeptID = Department.DeptID} Department$$

2.Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints

```
mysql> SELECT * FROM Department;
+--------+------------------------+
| DeptID | DeptName               |
+--------+------------------------+
|      5 | Civil                  |
|      1 | Computer Science       |
|      3 | Electronics            |
|      2 | Information Technology |
|      4 | Mechanical             |
+--------+------------------------+
5 rows in set (0.008 sec)

mysql> ^C
mysql> SELECT * FROM Student;
+-----+-------+------------+--------+
| SID | Name  | DOB        | DeptID |
+-----+-------+------------+--------+
| 101 | Rahul | 2004-03-15 |      1 |
| 102 | Priya | 2003-11-20 |      2 |
| 103 | Amit  | 2004-01-10 |      1 |
| 104 | Sneha | 2003-08-05 |      3 |
| 105 | Kiran | 2004-06-25 |      5 |
+-----+-------+------------+--------+
5 rows in set (0.019 sec)
```

3.Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

```
mysql> SELECT Product,COUNT(*) AS TotalSales
    -> FROM Sales
    -> GROUP BY Product;
+----------+------------+
| Product  | TotalSales |
+----------+------------+
| Laptop   |          3 |
| Mouse    |          2 |
| Keyboard |          2 |
| Monitor  |          1 |
+----------+------------+
4 rows in set (0.071 sec)

mysql> SELECT Product,SUM(Amount) AS TotalAmount
    -> FROM Sales
    -> GROUP BY Product;
+----------+-------------+
| Product  | TotalAmount |
+----------+-------------+
| Laptop   |   150000.00 |
| Mouse    |     2500.00 |
| Keyboard |     4500.00 |
| Monitor  |    12000.00 |
+----------+-------------+
4 rows in set (0.012 sec)

mysql> SELECT Product,AVG(Amount) AS AverageAmount
    -> FROM Sales
    -> GROUP BY Product;
+----------+---------------+
| Product  | AverageAmount |
+----------+---------------+
| Laptop   |  50000.000000 |
| Mouse    |   1250.000000 |
| Keyboard |   2250.000000 |
| Monitor  |  12000.000000 |
+----------+---------------+
4 rows in set (0.084 sec)

mysql> SELECT MAX(Amount) AS HighestSale
    -> FROM Sales;
+-------------+
| HighestSale |
+-------------+
|    55000.00 |
+-------------+
1 row in set (0.271 sec)

mysql> SELECT MIN(Amount) AS LowestSale
    -> FROM Sales;
+------------+
| LowestSale |
+------------+
|    1000.00 |
+------------+
1 row in set (0.012 sec)

mysql> SELECT Product,SUM(Amount) AS TotalAmount
    -> FROM Sales
    -> GROUP BY Product
    -> HAVING SUM(Amount)>50000;
+---------+-------------+
| Product | TotalAmount |
+---------+-------------+
| Laptop  |   150000.00 |
+---------+-------------+
```

4. Write a correlated sub-query to find employees earning more than the average salary of their own department.

```
mysql> SELECT *
    -> FROM Employee E
    -> WHERE Salary >
    -> (
    -> SELECT AVG(Salary)
    -> FROM Employee
    -> WHERE DeptID=E.DeptID
    -> );
+-------+---------+--------+--------+
| EmpID | EmpName | DeptID | Salary |
+-------+---------+--------+--------+
|   101 | Rahul   |      1 |  60000 |
|   104 | Sneha   |      2 |  50000 |
|   105 | Kiran   |      3 |  70000 |
+-------+---------+--------+--------+
3 rows in set (0.379 sec)
```

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

```
    -> ON Employee.ProjectID=Project.ProjectID;
+-------+---------+--------+--------+-----------+-----------+--------------+
| EmpID | EmpName | DeptID | Salary | ProjectID | ProjectID | ProjectName  |
+-------+---------+--------+--------+-----------+-----------+--------------+
|   101 | Rahul   |      1 |  60000 |         1 |         1 | AI Project   |
|   102 | Priya   |      2 |  45000 |         2 |         2 | Web Portal   |
|   103 | Amit    |      1 |  55000 |         1 |         1 | AI Project   |
|   104 | Sneha   |      2 |  50000 |         3 |         3 | Cloud System |
|   106 | Ravi    |      3 |  65000 |         2 |         2 | Web Portal   |
+-------+---------+--------+--------+-----------+-----------+--------------+
5 rows in set (0.010 sec)

mysql> SELECT *
    -> FROM Employee
    -> LEFT JOIN Project
    -> ON Employee.ProjectID=Project.ProjectID;
+-------+---------+--------+--------+-----------+-----------+--------------+
| EmpID | EmpName | DeptID | Salary | ProjectID | ProjectID | ProjectName  |
+-------+---------+--------+--------+-----------+-----------+--------------+
|   101 | Rahul   |      1 |  60000 |         1 |         1 | AI Project   |
|   102 | Priya   |      2 |  45000 |         2 |         2 | Web Portal   |
|   103 | Amit    |      1 |  55000 |         1 |         1 | AI Project   |
|   104 | Sneha   |      2 |  50000 |         3 |         3 | Cloud System |
|   105 | Kiran   |      3 |  70000 |      NULL |      NULL | NULL         |
|   106 | Ravi    |      3 |  65000 |         2 |         2 | Web Portal   |
+-------+---------+--------+--------+-----------+-----------+--------------+
6 rows in set (0.013 sec)

mysql> SELECT *
    -> FROM Employee
    -> RIGHT JOIN Project
    -> ON Employee.ProjectID=Project.ProjectID;
+-------+---------+--------+--------+-----------+-----------+--------------+
| EmpID | EmpName | DeptID | Salary | ProjectID | ProjectID | ProjectName  |
+-------+---------+--------+--------+-----------+-----------+--------------+
|   103 | Amit    |      1 |  55000 |         1 |         1 | AI Project   |
|   101 | Rahul   |      1 |  60000 |         1 |         1 | AI Project   |
|   106 | Ravi    |      3 |  65000 |         2 |         2 | Web Portal   |
|   102 | Priya   |      2 |  45000 |         2 |         2 | Web Portal   |
|   104 | Sneha   |      2 |  50000 |         3 |         3 | Cloud System |
|  NULL | NULL    |   NULL |   NULL |      NULL |         4 | Mobile App   |
+-------+---------+--------+--------+-----------+-----------+--------------+
6 rows in set (0.373 sec)

mysql> SELECT *
    -> FROM Employee
    -> LEFT JOIN Project
    -> ON Employee.ProjectID=Project.ProjectID
    ->
    -> UNION
    ->
    -> SELECT *
    -> FROM Employee
    -> RIGHT JOIN Project
    -> ON Employee.ProjectID=Project.ProjectID;
+-------+---------+--------+--------+-----------+-----------+--------------+
| EmpID | EmpName | DeptID | Salary | ProjectID | ProjectID | ProjectName  |
+-------+---------+--------+--------+-----------+-----------+--------------+
|   101 | Rahul   |      1 |  60000 |         1 |         1 | AI Project   |
|   102 | Priya   |      2 |  45000 |         2 |         2 | Web Portal   |
|   103 | Amit    |      1 |  55000 |         1 |         1 | AI Project   |
|   104 | Sneha   |      2 |  50000 |         3 |         3 | Cloud System |
|   105 | Kiran   |      3 |  70000 |      NULL |      NULL | NULL         |
|   106 | Ravi    |      3 |  65000 |         2 |         2 | Web Portal   |
|  NULL | NULL    |   NULL |   NULL |      NULL |         4 | Mobile App   |
+-------+---------+--------+--------+-----------+-----------+--------------+
7 rows in set (0.222 sec)

mysql> SELECT *
    -> FROM Employee
    -> CROSS JOIN Project;
+-------+---------+--------+--------+-----------+-----------+--------------+
| EmpID | EmpName | DeptID | Salary | ProjectID | ProjectID | ProjectName  |
+-------+---------+--------+--------+-----------+-----------+--------------+
|   101 | Rahul   |      1 |  60000 |         1 |         4 | Mobile App   |
|   101 | Rahul   |      1 |  60000 |         1 |         3 | Cloud System |
|   101 | Rahul   |      1 |  60000 |         1 |         2 | Web Portal   |
|   101 | Rahul   |      1 |  60000 |         1 |         1 | AI Project   |
|   102 | Priya   |      2 |  45000 |         2 |         4 | Mobile App   |
|   102 | Priya   |      2 |  45000 |         2 |         3 | Cloud System |
|   102 | Priya   |      2 |  45000 |         2 |         2 | Web Portal   |
|   102 | Priya   |      2 |  45000 |         2 |         1 | AI Project   |
|   103 | Amit    |      1 |  55000 |         1 |         4 | Mobile App   |
|   103 | Amit    |      1 |  55000 |         1 |         3 | Cloud System |
|   103 | Amit    |      1 |  55000 |         1 |         2 | Web Portal   |
|   103 | Amit    |      1 |  55000 |         1 |         1 | AI Project   |
|   104 | Sneha   |      2 |  50000 |         3 |         4 | Mobile App   |
|   104 | Sneha   |      2 |  50000 |         3 |         3 | Cloud System |
|   104 | Sneha   |      2 |  50000 |         3 |         2 | Web Portal   |
|   104 | Sneha   |      2 |  50000 |         3 |         1 | AI Project   |
|   105 | Kiran   |      3 |  70000 |      NULL |         4 | Mobile App   |
|   105 | Kiran   |      3 |  70000 |      NULL |         3 | Cloud System |
|   105 | Kiran   |      3 |  70000 |      NULL |         2 | Web Portal   |
|   105 | Kiran   |      3 |  70000 |      NULL |         1 | AI Project   |
|   106 | Ravi    |      3 |  65000 |         2 |         4 | Mobile App   |
|   106 | Ravi    |      3 |  65000 |         2 |         3 | Cloud System |
|   106 | Ravi    |      3 |  65000 |         2 |         2 | Web Portal   |
|   106 | Ravi    |      3 |  65000 |         2 |         1 | AI Project   |
+-------+---------+--------+--------+-----------+-----------+--------------+
24 rows in set (0.015 sec)
```

6. Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

```
mysql> SELECT *
    -> FROM DeptSalaryView;
+------------------------+----------------+---------------+
| DeptName               | TotalEmployees | AverageSalary |
+------------------------+----------------+---------------+
| Computer Science       |              2 |    57500.0000 |
| Information Technology |              2 |    47500.0000 |
| Electronics            |              2 |    67500.0000 |
+------------------------+----------------+---------------+
3 rows in set (0.216 sec)
```

7. Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

```
mysql> Select * From Enrollment;
+-----+-----+
| SID | CID |
+-----+-----+
| 101 |   1 |
| 102 |   1 |
| 103 |   1 |
| 104 |   1 |
| 101 |   2 |
| 103 |   2 |
| 105 |   2 |
| 101 |   3 |
| 102 |   3 |
| 103 |   3 |
+-----+-----+
10 rows in set (0.011 sec)
```

8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

```
mysql> Select * From Student;
+-----+-------+------------+--------+
| SID | Name  | DOB        | DeptID |
+-----+-------+------------+--------+
| 101 | Rahul | 2004-03-15 |      1 |
| 102 | Priya | 2003-11-20 |      2 |
| 103 | Amit  | 2004-01-10 |      1 |
| 104 | Sneha | 2003-08-05 |      3 |
| 105 | Kiran | 2004-06-25 |      5 |
+-----+-------+------------+--------+
5 rows in set (0.008 sec)
```

9. Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

```
+-------+---------+--------+--------+-----------+
| EmpID | EmpName | DeptID | Salary | ProjectID |
+-------+---------+--------+--------+-----------+
|   101 | Rahul   |      1 |  60000 |         1 |
|   103 | Amit    |      1 |  55000 |         1 |
+-------+---------+--------+--------+-----------+
2 rows in set (0.009 sec)
```

10. Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

```
mysql> DESC Flight;
+-------------+---------------+------+-----+---------+-------+
| Field       | Type          | Null | Key | Default | Extra |
+-------------+---------------+------+-----+---------+-------+
| FlightID    | int           | NO   | PRI | NULL    |       |
| FlightName  | varchar(50)   | YES  |     | NULL    |       |
| Source      | varchar(50)   | YES  |     | NULL    |       |
| Destination | varchar(50)   | YES  |     | NULL    |       |
| Fare        | decimal(10,2) | YES  |     | NULL    |       |
+-------------+---------------+------+-----+---------+-------+
5 rows in set (0.017 sec)

mysql> Select * From Flight;
+----------+------------+-----------+-------------+---------+
| FlightID | FlightName | Source    | Destination | Fare    |
+----------+------------+-----------+-------------+---------+
|      101 | IndiGo     | Chennai   | Delhi       | 5500.00 |
|      102 | Air India  | Hyderabad | Mumbai      | 4800.00 |
|      103 | SpiceJet   | Bangalore | Kolkata     | 6200.00 |
+----------+------------+-----------+-------------+---------+
3 rows in set (0.006 sec)
```